



UNIVERSITÉ
LAVAL

Processus de génie logiciel

GLO-2003

Hiver 2026

9. Agilité et Extreme programming

Constats sur le monde des affaires

- Le monde des affaires opère maintenant dans un environnement globalisé où:
 - 1) le logiciel est omniprésent;
 - 2) la vitesse de réponse aux opportunités et aux conditions changeantes devient cruciale.
- En raison de cet environnement d'affaires en mouvance, il devient souvent impossible d'obtenir un ensemble de requis stables. Il faut parfois une première version du logiciel avant de comprendre tous les besoins des utilisateurs.

Développement logiciel rapide

- Le besoin de méthodes pour un développement logiciel rapide est connu depuis 1980. Il s'agit de développer du logiciel utile rapidement.
 - Les requis, la conception et l'implémentation sont entremêlés. On ne détaille pas les spécifications du système et la documentation est minimisée ou générée automatiquement.
 - Le système est développé en séries de versions. Les parties prenantes sont constamment impliquées dans l'évaluation des versions. Leurs commentaires sont utilisés dans le développement.
 - L'interface utilisateur est développée rapidement.

Développement logiciel rapide

- Les approches disciplinées suggèrent des pratiques pour cerner les exigences et développer le **bon** logiciel.
- L'effort de planification doit être plus substantiel. Ceci ralentit le développement.
- Cette approche est essentielle pour la réussite de certains projets, mais problématique là où la vitesse de développement est critique.

Raison d'être des processus disciplinés

- Dans la première décennie du millénaire, les meilleures pratiques de développement qui faisaient consensus étaient:
 - Une planification adaptative, mais méticuleuse
 - Une assurance qualité formalisée
 - L'utilisation de méthodes d'analyse et conception supportés par des outils CASE
 - Un processus contrôlé et rigoureux

Le coût des approches disciplinées

- Les approches disciplinées, à moins d'être judicieusement adaptées, impliquent un effort important de planification, de conception et de documentation.
- Pour les petits et moyens projets, cet effort devient trop important relativement à l'effort total de développement et peut même dominer le processus.

Origine de l'agilité

Le 13 février
2001, dix-sept
personnes
rédigent autour
d'une bonne
table:

Agile software
development
manifesto



Les valeurs de l'Agilité selon le manifeste

- **Les individus et les interactions**
priment sur les processus et les outils
- **Les logiciels opérationnels**
priment sur la documentation comprehensive
- **La collaboration avec le client**
priment sur la négociation d'un contrat
- **L'adaptation aux changements**
priment sur le suivi d'un plan



Les principes de l'Agilité selon le manifeste

1) Satisfaire le client est la priorité



2) Accueillir les demandes de changement



3) Livrer le plus souvent possible des versions opérationnelles de l'application



Les principes de l'Agilité selon le manifeste

- 4) Assurer une coopération permanente entre le client et l'équipe du projet
- 5) Construire des projets autour d'individus motivés
- 6) Privilégier la conversation en face à face



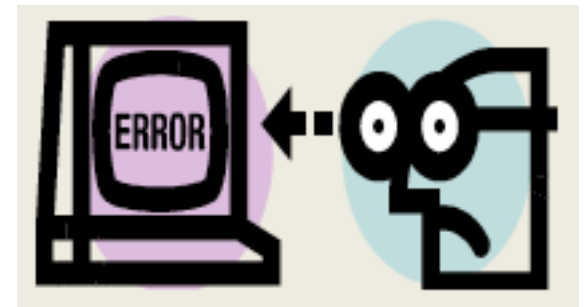
Les principes de l'Agilité selon le manifeste

7) Mesurer l'avancement du projet en termes de fonctionnalités de l'application

8) Faire avancer le projet à un rythme soutenable et constant



9) Porter une attention continue à l'excellence technique et à la conception

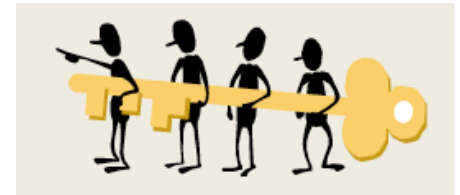


Les principes de l'Agilité selon le manifeste

10) Favoriser la simplicité



11) Responsabiliser les équipes: les meilleures architectures, spécifications et conceptions émergent d'équipes auto-organisées.



12) Ajuster, à intervalles réguliers, son comportement pour être plus efficace

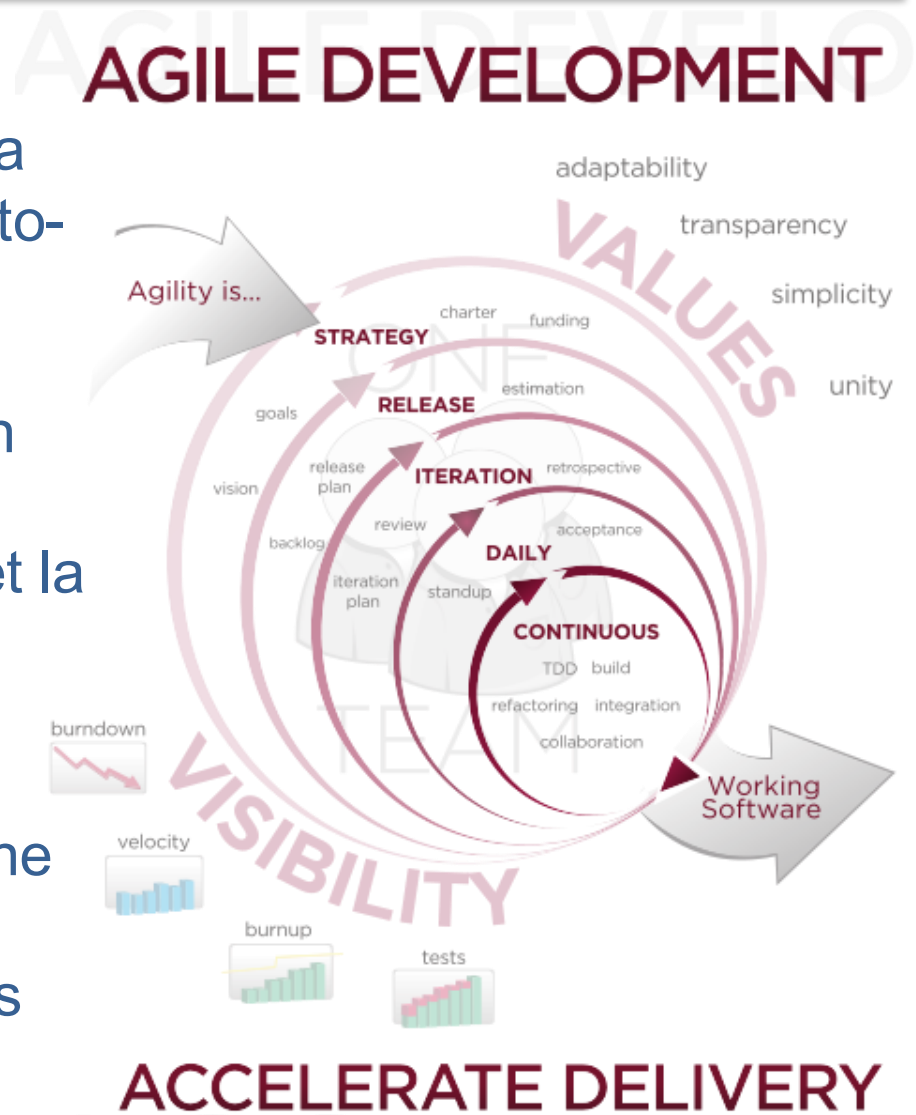


Développement agile de logiciels

En développement logiciel, les *pratiques agiles* mettent en avant la collaboration entre des équipes auto-organisées et pluridisciplinaires et leurs clients.

Elles s'appuient sur l'utilisation d'un cadre méthodologique léger mais suffisamment centré sur l'humain et la communication.

Elles préconisent une planification adaptative, un développement évolutif, une livraison précoce et une amélioration continue, et elles encouragent des réponses flexibles au changement



Source de l'image: www.versionone.com

Panorama agile

- eXtreme Programming (XP, 1999)
- Dynamic Software Development Method (DSDM, 1995)
- Crystal (2004)
- Feature Driven Development (FDD, 1999)
- Lean Software Development (2003)
- Test Driven Development (TDD, 2002)
- Kanban
- SCRUM (1996)
- SAFe (2007)
- DevOps (2009)
- Disciplined agile delivery (2012)



Agile vs. Discipliné

Agile vs Discipliné

- Formalisme ne signifie pas Discipline
- Documentation ne signifie pas Communication
- Documentation ne signifie pas Compréhension
- Rigueur ne signifie pas Qualité

But : Augmenter le niveau de **satisfaction** des clients tout en rendant plus **facile** le travail des développeurs

Synthèse des principes agiles

- ✓ Implication du client
- ✓ Livraison incrémentale
- ✓ Emphase sur les développeurs, pas le processus
- ✓ Voir le changement comme une opportunité
Embrace change!
- ✓ Maintenir la simplicité

Prétentions de l'agilité

- Le développement agile est exécuté dans un esprit de **collaboration**.
- Le développement agile répond aux **besoins changeants** des parties prenantes.
- Le produit logiciel est de **haute qualité**
- Bon rapport **coût-efficacité** en **temps voulu**.
- Les équipes sont **auto-organisées**

Les méthodes agiles

➤ **Succès** significatifs de ces méthodes pour:

- Le développement de **petits** ou **moyens** produits destinés à la vente.
- Des **applications maison** pour une organisation où un « **client** » **s'implique** de façon soutenue et qui sont assujetties à peu de normes ou règlements externes.

Les méthodes agiles

➤ Difficultés d'application:

- La **collaboration active** du client est difficile à obtenir.
- Lorsque l'équipe de développement souffre d'un **manque de cohésion** entre ses membres.
- La difficulté à maintenir **la continuité de l'équipe** de développement.
- Lorsqu'il y a **plusieurs intervenants** et que ceux-ci priorisent des requis différents.

Les méthodes agiles

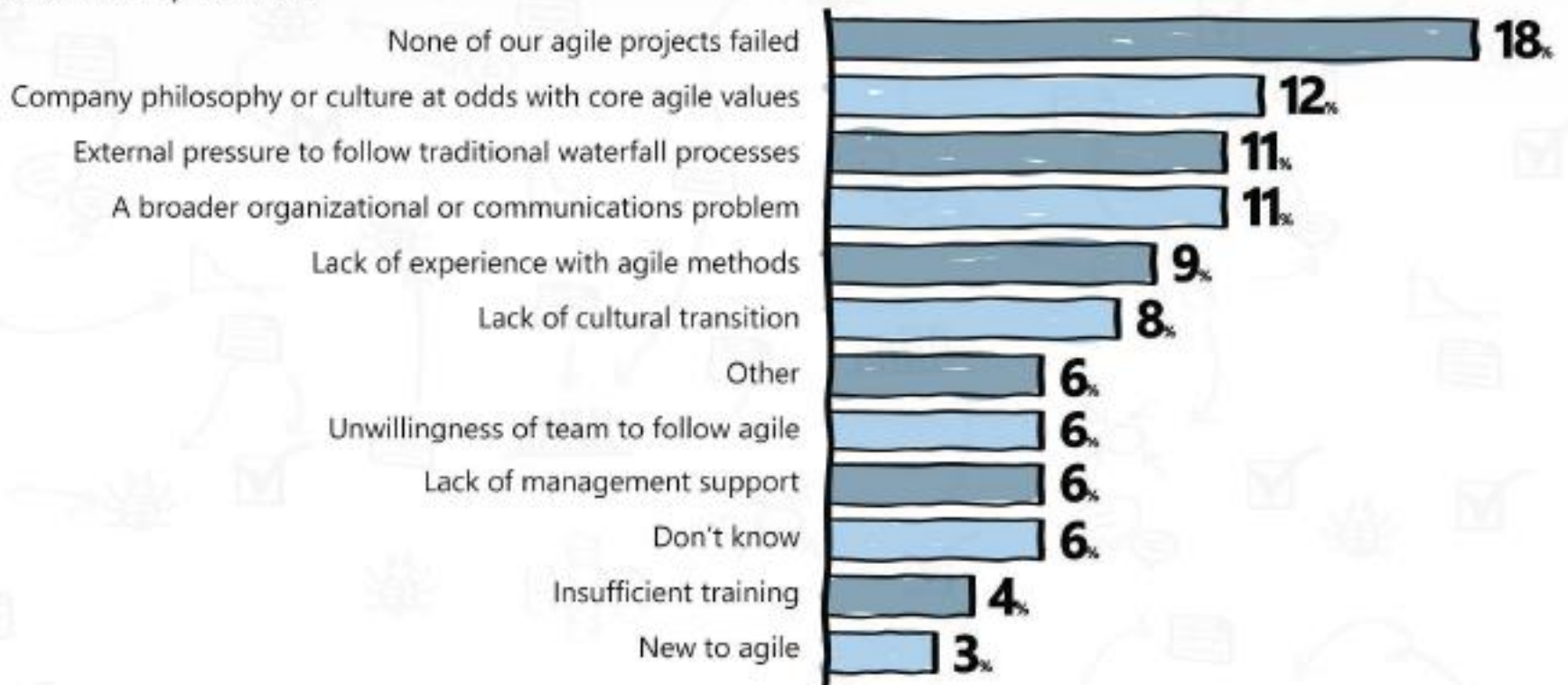
➤ Difficultés d'application:

- **Maintenir la simplicité** demande davantage de travail.
- Les grandes organisations ont mis des années à implanter un processus structuré. Elles sont **résistantes au changement**.
- Les cycles d'évolution lors de projet de maintenance seront plus coûteux puisque **l'absence de documentation** ralentit le travail (dans le cas du document de spécification en particulier).

Les méthodes agiles

LEADING CAUSES OF FAILED AGILE PROJECTS

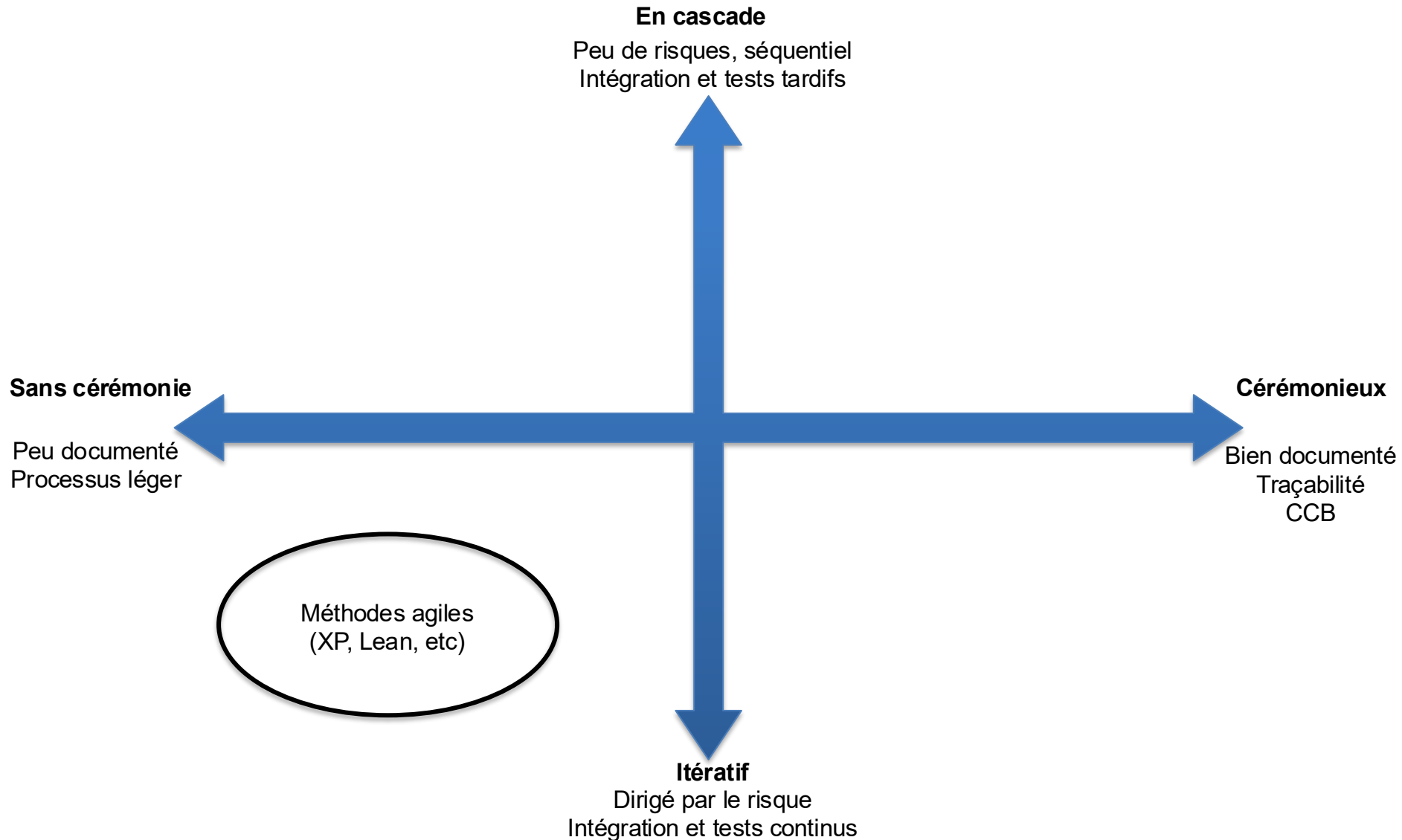
Most respondents said none of their agile projects would be considered unsuccessful (18%). Of those with failed agile projects, most said it was due to either a company philosophy or culture at odds with core agile values (12%), external pressure to follow waterfall processes (11%), or a broader organizational or communications problem (11%).



Êtes-vous agiles?

			1	2	3	4	5	
Team Communication	1	minimal, written, knowledge is power	☐	☐	☐	☐	☐	open, trusting, face-to-face
User Accessibility	2	limited, off-site	☐	☐	☐	☐	☐	constant, on-site
Team Location	3	highly distributed	☐	☐	☐	☐	☐	co-located
Team Structure	4	departmental, top down, large teams	☐	☐	☐	☐	☐	cross-functional, self-organizing, small teams
Delivery Frequency (Shippable)	5	infrequent, 3+ months	☐	☐	☐	☐	☐	frequent, 1-2 weeks
Measurement of Progress	6	phases, tasks, documents	☐	☐	☐	☐	☐	features/business value, working software
Ability to Change Direction	7	low, prevented	☐	☐	☐	☐	☐	high, embraced
Testing	8	manual, post-coding	☐	☐	☐	☐	☐	integrated, automated, test-driven
Planning Approach	9	up-front, detailed, activity-based	☐	☐	☐	☐	☐	just enough, adaptive, continuous
Process Philosophy	10	static, audited, "my-way"	☐	☐	☐	☐	☐	analyze/adapt/improve

Carte des processus logiciel

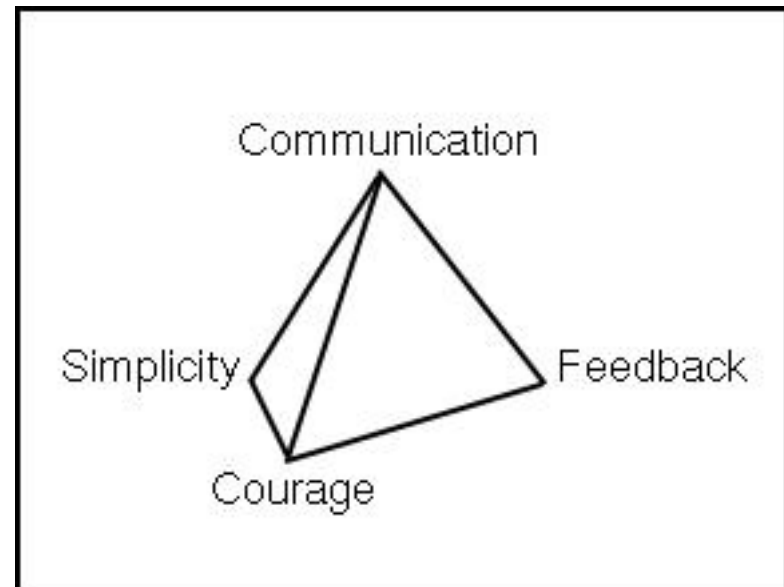


eXtreme Programming

XP, une des premières méthodes agiles qui propose de pousser à l'extrême les bonnes pratiques de programmation.

Les valeurs XP

- Communication
- Simplicité
- Rétroaction
- Courage
- Respect (2004)



Droits du client

- Vous avez le droit de changer d'avis, de substituer des fonctionnalités et de changer des priorités.
 - Pour maximiser votre investissement.
- Vous avez le droit de voir l'avancement du produit, prouvé par des tests d'acceptation répétables.
- Vous avez le droit d'être informé de changements de planification à temps pour choisir les solutions pour respecter vos échéances. Vous pouvez même annuler à tout moment le développement du produit et garder le résultat du travail reflétant l'investissement jusqu'à date.

Devoirs du client

- Faire confiance aux décisions techniques des développeurs
- Analyser les risques des récits adéquatement
- Choisir les récits ayant le plus d'impact
- Fournir des récits précis
- Travailler au sein de l'équipe

Droits du développeur

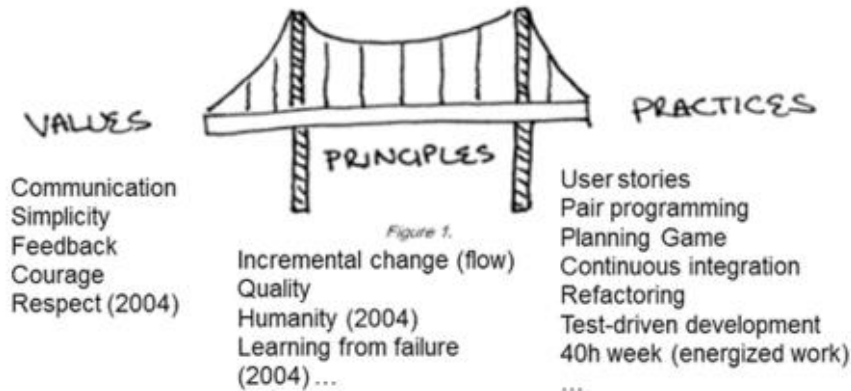
- Vous avez le droit de dire combien de temps chaque récit vous prendra pour l'implémenter ainsi que de réviser les estimations.
- Laisser au client les décisions d'affaire.
- Vous avez le droit de produire un produit de qualité à tout moment, qui satisfait les besoins du client.
- Vous avez le droit à un travail productif, agréable et amusant, avec un horaire prévisible et sensé.

Devoirs du développeur

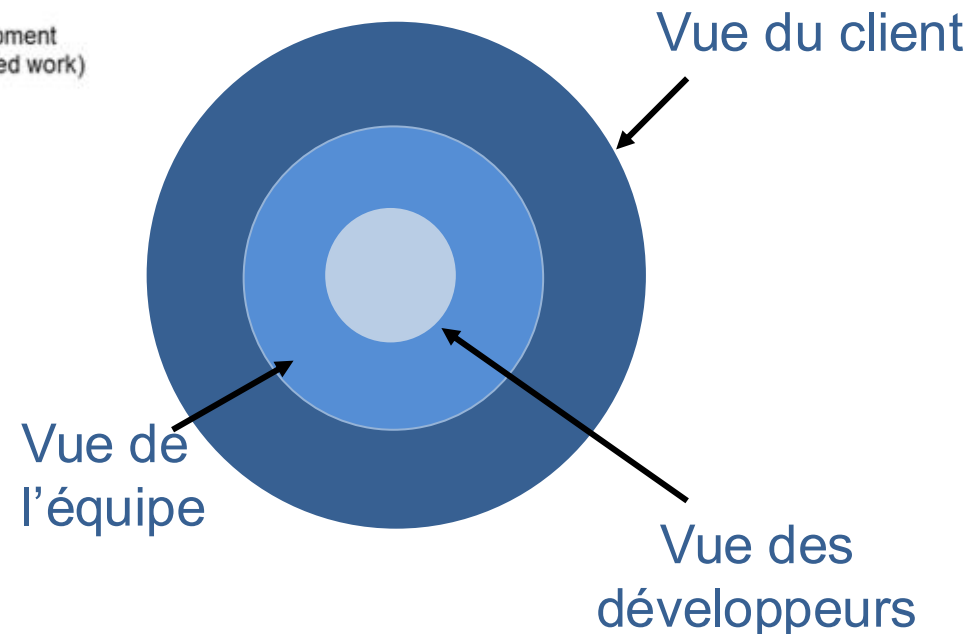
- Suivre les directives de l'équipe pour que le système soit simple et bien testé.
- Développer uniquement ce qui est demandé pour garder le projet simple et de grande valeur pour le client.
- Communiquer constamment avec le client pour comprendre ses enjeux et lui fournir l'information nécessaire à ses décisions.

eXtreme Programming

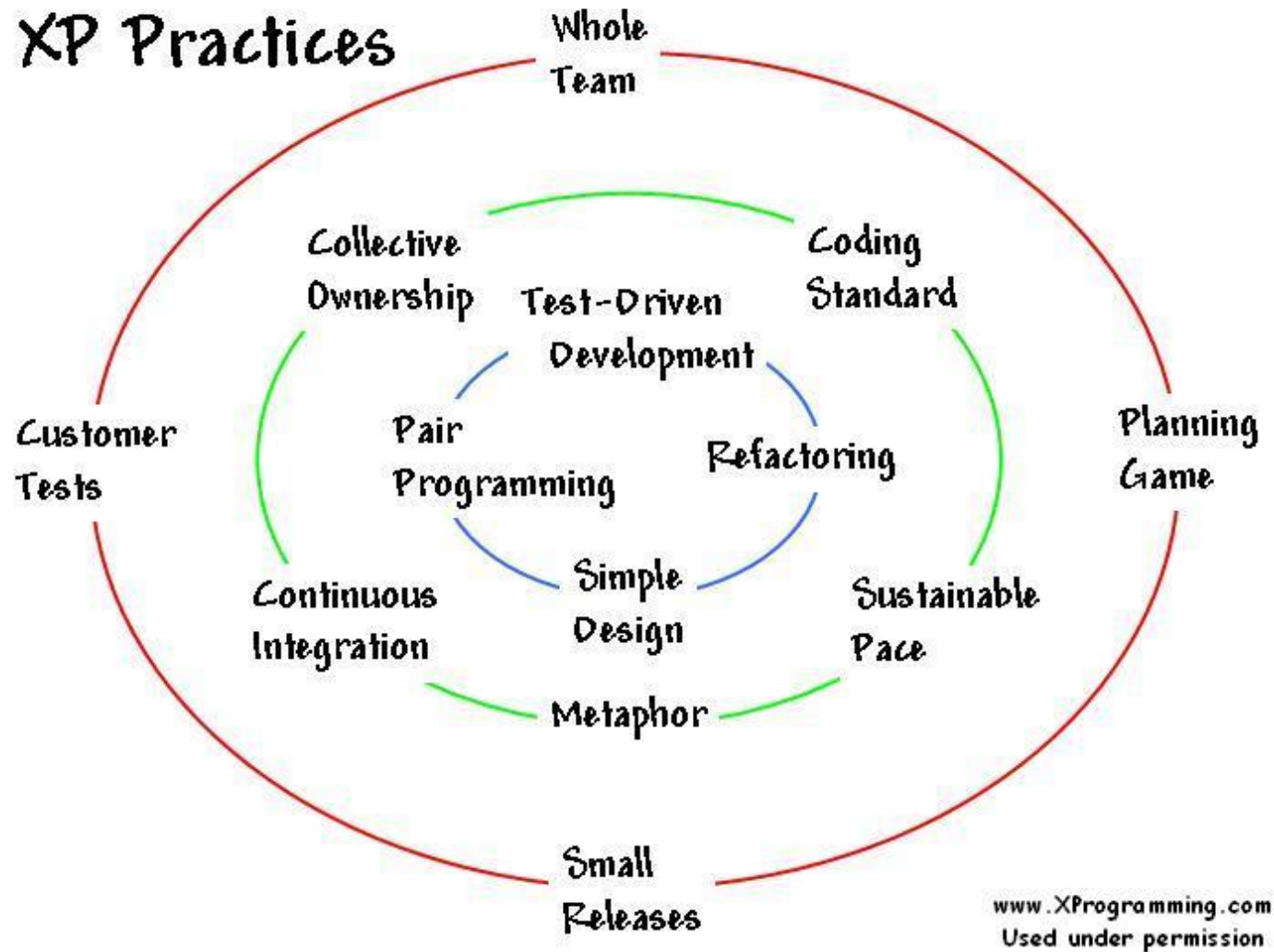
XP values, principles & practices



E.g. Communication feedback => humanity => pair programming



eXtreme Programming



Vue Client :



➤ Équipe tout entière

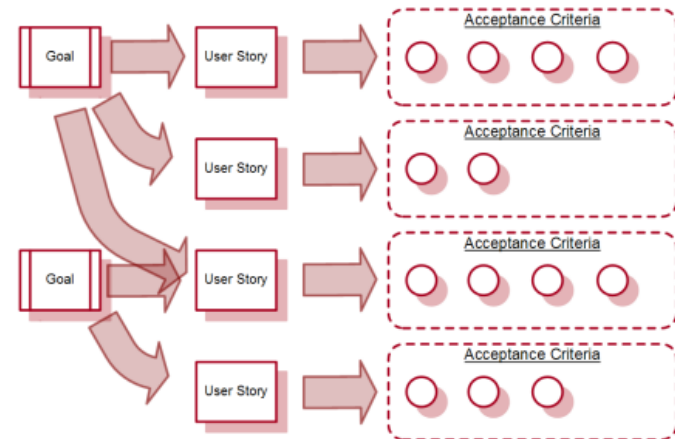
- **S'asseoir ensemble** – On travaille dans un espace ouvert qui peut contenir toute l'équipe.
 - **Client présent** – Le client est assis avec l'équipe à plein temps. C'est un membre de l'équipe.
 - **Espace de travail ouvert et informatif** – L'espace de travail devrait être consacré au projet.
- Avantages: progrès visible, agilité, réduit les problèmes de communication, accélère la communication.

Vue Client :

➤ Tests client

- **Récit utilisateur** – Une fonctionnalité que le client désire avoir.
- **Tests client** – Tests conçus par le client qui lui permettent de vérifier le progrès du développement.

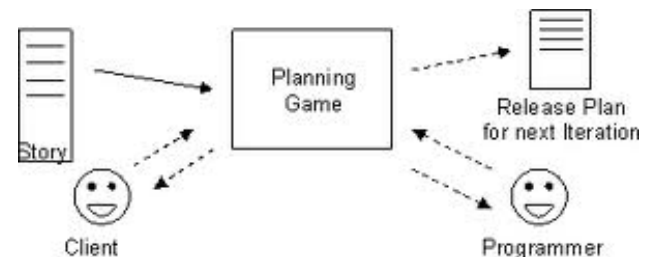
➤ Avantages: progrès tangible et vérifiable, traçabilité, répétabilité.



Pratiques XP

Vue Client :

- **Jeu de planification** - Le client décide le but et la planification des livraisons en se basant sur les évaluations fournies par des développeurs. Ceux-ci font l'implémentation seulement de la fonctionnalité exigée par les récits de cette itération.
- **Avantages:** Équipe autogérée, feedback rapide, objectifs à long-terme et court-terme, multiplie les occasions pour diriger l'équipe vers un projet réussi.



Pratiques XP

Vue Équipe :

- **Intégration continue** - Le nouveau code est intégré avec le système très rapidement. Tous les tests doivent être réussis sinon les fonctionnalités sont éliminées.
- **Propriété collective** - Les programmeurs améliorent n'importe quelle section du code, n'importe où dans le système, à tout moment, s'ils en voient l'occasion.

Pratiques XP

Vue Équipe :

- **Métaphores** - Le système est défini par une métaphore ou par un jeu de métaphores partagées entre le client et les programmeurs. Une métaphore est une histoire que chacun peut raconter en se référant au système.
- **Standard de programmation** – Pour faciliter la compréhension du code.
- **40-heures semaine (rythme soutenable)** - Personne ne peut travailler d'heures supplémentaires.

Vue Programmeur

- **Développement piloté par les tests (TDD)** - Les programmeurs écrivent les tests unitaires avant de coder. Les clients écrivent les tests fonctionnels pour les récits de l'itération.
- **Programmation en paire** - Tout le code de production est écrit par deux personnes à un écran / clavier / souris

Pratiques XP

Vue Programmeur

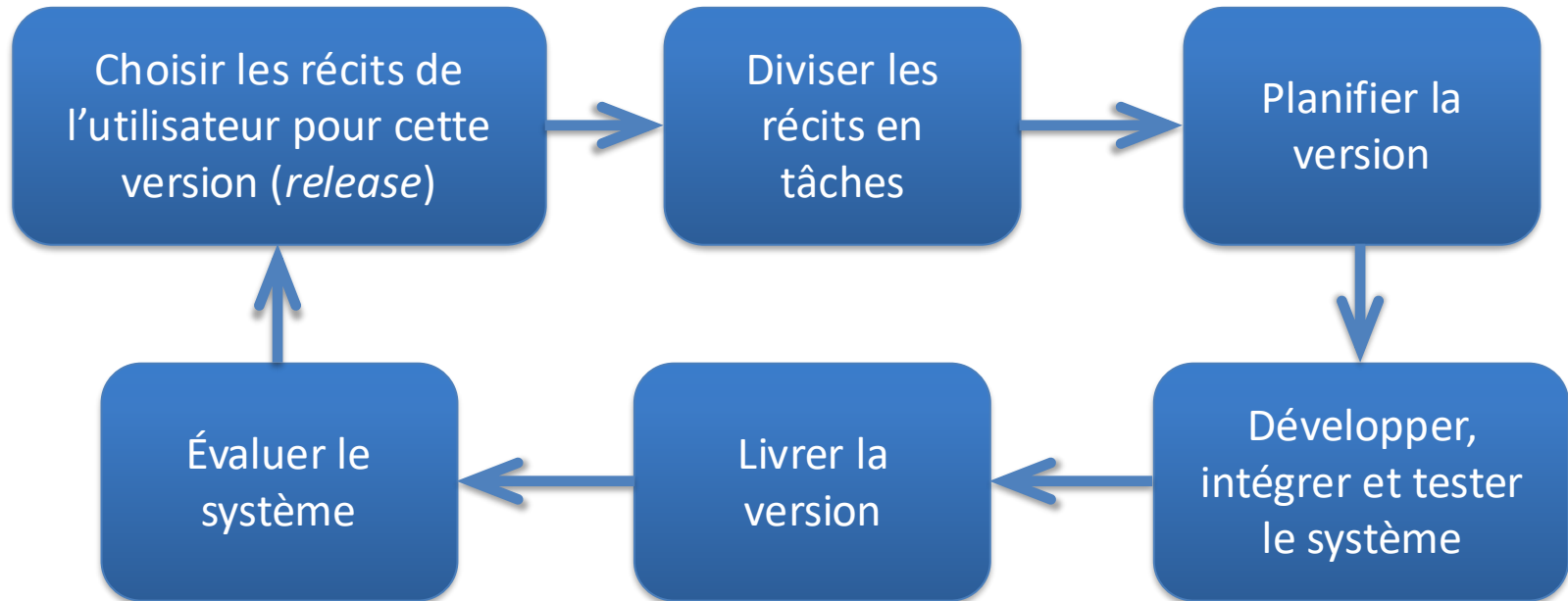
- **Réusinage** - La technique de transformation de code pour l'améliorer sans modifier le comportement. Cette pratique est concourante à l'implémentation.
- **Conception simple** – On fait juste assez de design pour implémenter les requis de l'itération courante. Pas plus. On reporte les décisions pour lesquelles on n'a pas d'information.

eXtreme Programming

L'unicité de XP

- Le client définit les caractéristiques de l'application
- L'équipe = développeurs + client
- Cycles de feedback (client → programmeur)
- Prescriptive au niveau de (10) minutes
- Conception émergente

Cycle de développement d'XP



XP: exemple de récit utilisateur

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either 'current medication', 'new medication' or 'formulary'.

If you select 'current medication', you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose 'new medication', the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose 'formulary', you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click 'OK' or 'Change'. If you click 'OK', your prescription will be recorded on the audit database. If you click 'Change', you reenter the 'Prescribing medication' process.

Selon les guidelines de Mike Cohn, ceci est un mauvais exemple de récit utilisateur.

Récit: En tant que « *qui* », je veux « *quoi* » afin de « *pourquoi* ».

XP: exemple de tâche

Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary id for the generic drug name, lookup the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.

Lors de la planification de l'itération, on divise le récit en tâches pour les développeurs en vue d'estimer l'effort d'implémentation.

XP: exemple de cas de test

Test 4: Dose checking

Input:

1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:

1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:

OK or error message indicating that the dose is outside the safe range.

L'écriture des tests d'acceptation avant l'implémentation comble le besoin de spécifications détaillées.

Problèmes avec le TDD

- La pratique de développement piloté par les tests ne mène pas nécessairement à une couverture complète de tests.
 1. Les programmeurs préfèrent coder que tester. Ils ont tendance à prendre des raccourcis lorsqu'ils écrivent leurs tests.
 2. Certains tests sont difficiles à concevoir incrémentalement. Certains tests de système pour des requis qui englobent des propriétés globales du système ne sont pas identifiés par cette approche.
 3. Il est difficile d'évaluer la complétude d'un ensemble de tests sans perspective globale. Beaucoup de tests ne signifie pas des tests complets.

Pratique de la programmation par paire

- Sous XP, les paires de programmation sont constituées dynamiquement, ce qui encourage les interactions multiples dans l'équipe.
- Les avantages sont:
 - La propriété collective du code est encouragée. *Egoless programming*. Chacun se sent responsable du code.
 - Revue informelle du code. Moins efficace que les revues formelles, mais abordable et facile à organiser.
 - Ça supporte le *refactoring*, ce qui améliore la qualité du logiciel.

Choisir entre discipliné et agile

Quelle approche choisir ?

- La plupart des éléments d'agilité sont maintenant incontournables.
- La réussite d'un projet dépend avant tout de l'adaptation de l'approche au contexte.
- L'agilité est une philosophie de travail qui s'applique dans le cadre d'un processus.

Choisir entre discipliné et agile

- Critères de décision (3 aspects à considérer: technique, humain et organisationnel)
 - Des spécifications et un design détaillés sont-ils importants? Si oui -> **approche disciplinée**
 - Est-ce qu'une stratégie de développement incrémental, avec plusieurs livraisons et une rétroaction rapide du client, est réaliste? Si oui -> **approche agile**
 - Quelle est la taille du système à développer? Les approches agiles sont particulièrement efficaces pour les petits systèmes développés localement par une seule équipe. Grand projet, équipe nombreuse -> **approche disciplinée**

Choisir entre discipliné et agile

- Quel est le type du système développé? Faut-il une analyse & conception poussée avant l'implémentation? Si oui -> ~~approche disciplinée~~
- Quelle est la durée de vie prévue du système? Un système prévu pour durer nécessitera probablement une documentation plus exhaustive. Si oui -> ~~approche disciplinée, à condition que la documentation soit tenue à jour~~
- Est-ce que des technologies sont disponibles pour supporter le développement logiciel? Si vous développez un système en utilisant un IDE rudimentaire, plus de documentation de conception est suggérée -> ~~approche disciplinée~~

Choisir entre discipliné et agile

- Quelle est l'organisation de l'équipe? Si votre contexte nécessite de la communication inter-équipe (distribuée, out-sourcing) -> **approche disciplinée**
- Y a-t-il des facteurs de culture organisationnelle ayant un impact sur le développement?
- Quelles sont les compétences des développeurs? Une présence significative de programmeurs avec peu d'expérience ou moyennement compétents, l'approche agile peut être problématique.
- Le système sera-t-il soumis à des réglementations du domaine d'application? Une documentation détaillée est habituellement demandée pour les systèmes critiques -> **approche disciplinée**

Suppositions des approches agiles

- En utilisant les approches agiles, on suppose que:
- Le client est au même endroit que les développeurs et disponible pour répondre aux besoins du projet.
 - La documentation et les modèles ne jouent pas de rôle prépondérant dans le développement logiciel.
 - Les exigences évoluent en même temps que le logiciel.
 - Un processus de développement qui s'adapte dynamiquement à un projet et à un produit aux caractéristiques instables a de meilleures chances de produire du logiciel de qualité.
 - Les développeurs ont l'expérience appropriée pour définir et adapter eux-mêmes leur processus de développement.

Suppositions des approches agiles

- Les organisations supportent la formation d'équipes formées de développeurs doués et brillants, expérimentés dans la résolution de problèmes et capables d'améliorer eux-mêmes leur processus.
- L'avancement d'un projet se mesure essentiellement en termes de livraisons, et non de métriques.
- Une évaluation rigoureuse des artefacts peut se limiter à des revues informelles et des tests du code.
- L'écriture de code réutilisable et général n'est pas un objectif majeur de développement de logiciel.
- Le coût d'un changement ne croît pas avec le temps.
- Le logiciel peut être développé par incréments.
- Il est inutile de concevoir le logiciel pour le changement. Le refactoring peut s'en occuper.

L'agilité et les grands projets

- Différences entre petits et grands projets:
 - Les grands systèmes sont habituellement développés en les séparant en sous-systèmes, chacun étant développé par des équipes différentes.
 - Les grands systèmes interagissent avec ou incluent habituellement des systèmes existants. Les requis qui touchent ces interactions ne sont pas facilement flexibles ou adaptables au développement incrémental.

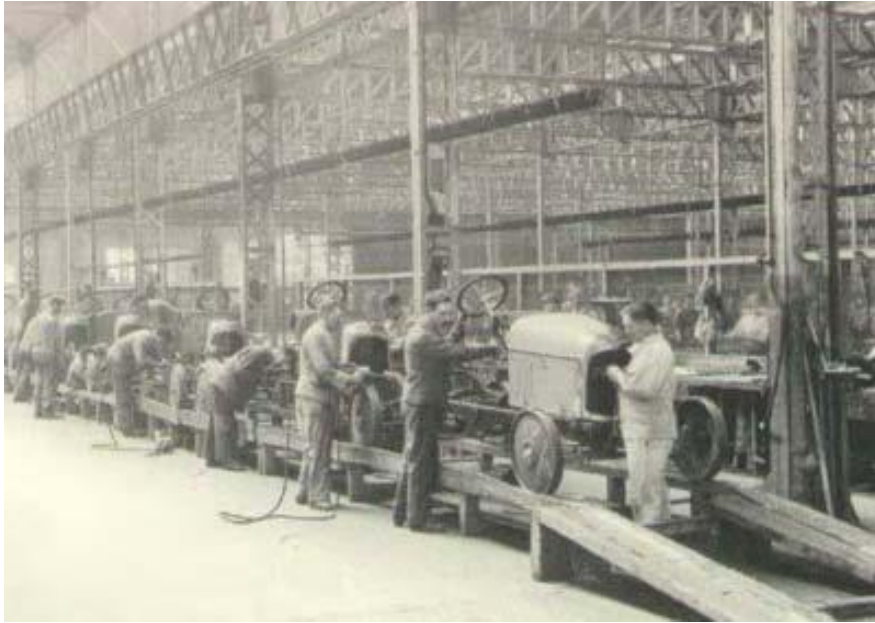
L'agilité et les grands projets

- Différences entre petits et grands projets:
 - Les grands systèmes sont plus souvent l'objet de réglementations externes.
 - Les projets de développement de grands systèmes sont généralement de longue durée. Il est difficile de maintenir la cohérence d'une équipe pour de longue période.
 - Les grands projets ont souvent un ensemble d'utilisateurs différents. Il est impossible de tous les impliquer.

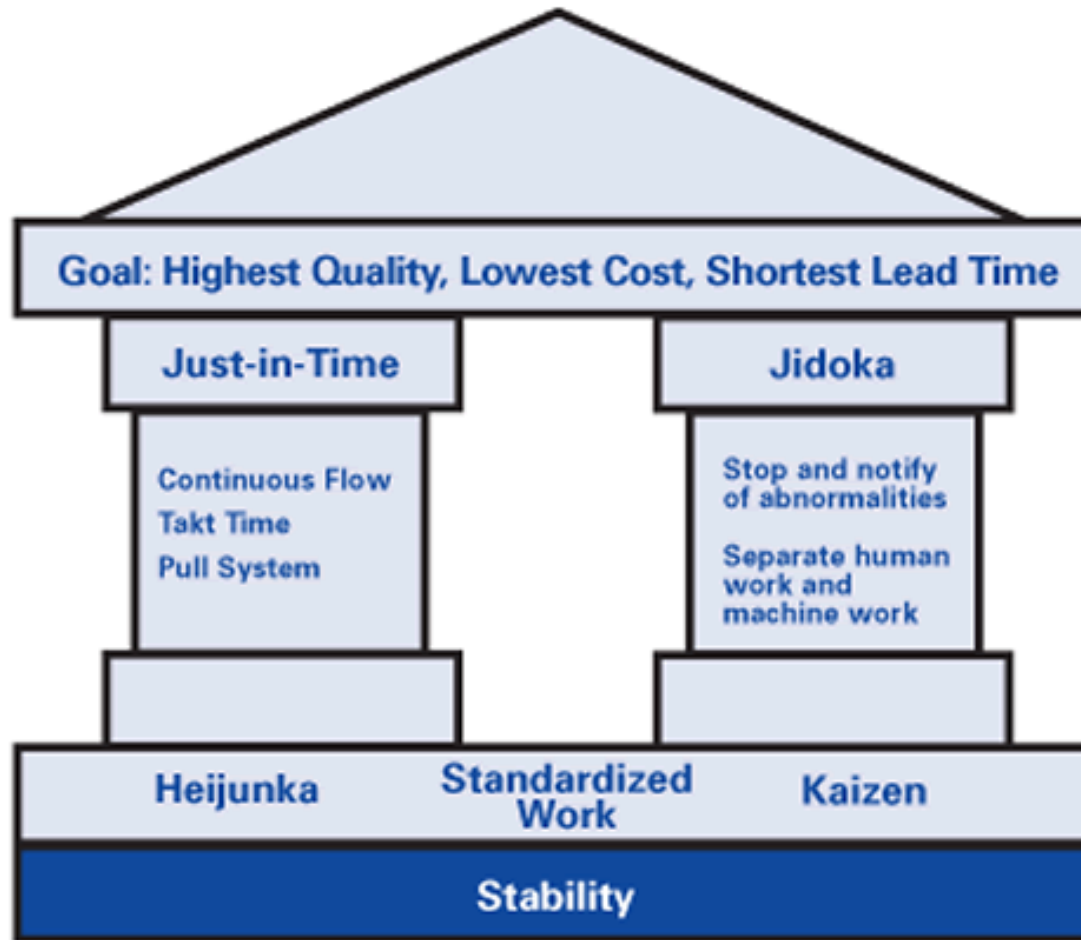
L'agilité et les grands projets

- Adaptations suggérées pour les grands projets:
 - Introduire davantage d'activités de conception et produire de la documentation sur les aspects critiques du système.
 - Établir des mécanismes de communication entre les équipes et s'assurer que les canaux restent ouverts.

Développement logiciel et le taylorisme



Développement logiciel et la méthode Toyota



Toyota Production System "House."

Conclusion

- Les supporteurs de méthodes agiles ont souvent fait la promotion de leur approche de façon évangélique. La réponse de certains tenants des approches disciplinées a aussi été extrême.
- En réalité, l'étiquette d'un projet a peu d'importance pourvu que le logiciel soit de qualité et que le client soit satisfait.
- Les approches hybrides sont souvent les plus performantes. La compatibilité dépend des valeurs et non des pratiques.
- Plusieurs compagnies qui affirment faire du développement agile ont plutôt adopté des pratiques agiles et les ont intégrées dans leur processus discipliné.